

ViduraAI - Complete Project Documentation

Table of Contents

1. [Project Overview](#)
 2. [Tech Stack](#)
 3. [System Architecture](#)
 4. [Core Components](#)
 5. [Backend Implementation](#)
 6. [Frontend Implementation](#)
 7. [Database Schema](#)
 8. [API Endpoints](#)
 9. [Key Functions & Methods](#)
 10. [Data Flow](#)
 11. [Configuration](#)
 12. [Installation & Setup](#)
-

Project Overview

ViduraAI is a sophisticated dual-mode conversational AI platform that combines general chatbot capabilities with advanced document-based question answering through Retrieval-Augmented Generation (RAG).

Key Features

- **Dual-Mode Intelligence:** Switch between general conversation and document-specific queries
 - **Advanced RAG Pipeline:** Semantic chunking, vector search, and context-aware retrieval
 - **Persistent Memory:** PostgreSQL-backed session management with full conversation history
 - **Local LLM Processing:** Privacy-focused with LLaMA 3.2 1B models
 - **Modern Web Interface:** Real-time streaming responses with ChatGPT-inspired UI
 - **Smart Routing:** Automatic fallback to general mode for low-confidence queries
-

Tech Stack

Backend Framework

- **Flask 3.0+**: Web framework for routing and API endpoints
- **Python 3.8+**: Core programming language

Language Models

- **LLaMA 3.2 1B Instruct**: Quantized models (Q5_K_L) for inference
 - Document Model: Specialized for document-based responses
 - General Model: Optimized for open-ended conversations
- **llama-cpp-python**: Python bindings for efficient LLaMA inference

Vector Database & Embeddings

- **ChromaDB 0.4.22**: Vector database for semantic search
- **E5-Large-v2**: Embedding model (sentence-transformers 3.0.1)
- **HuggingFace Transformers 4.36.2**: Model loading and inference

Document Processing

- **LangChain 0.1.0**: Document processing framework
- **LangChain-Community 0.0.13**: Community integrations
- **PyPDF 6.0.0 / PyPDF2 3.0.1**: PDF document parsing
- **python-docx 1.1.0**: Word document processing
- **PyMuPDF**: Advanced PDF extraction
- **docx2txt 0.9**: Text extraction from DOCX

Database & Persistence

- **PostgreSQL 12.0+**: Relational database for conversation history
- **SQLAlchemy**: ORM for database operations
- **LangChain-Postgres**: PostgreSQL integration for LangChain
- **psycopg[binary]**: PostgreSQL adapter

Machine Learning & NLP

- **torch 2.1.2**: PyTorch for deep learning
- **accelerate 0.25.0**: Efficient model loading
- **bitsandbytes 0.42.0**: Model quantization
- **NLTK**: Natural language processing toolkit
- **numpy 1.24.3**: Numerical computing
- **pandas 2.1.4**: Data manipulation

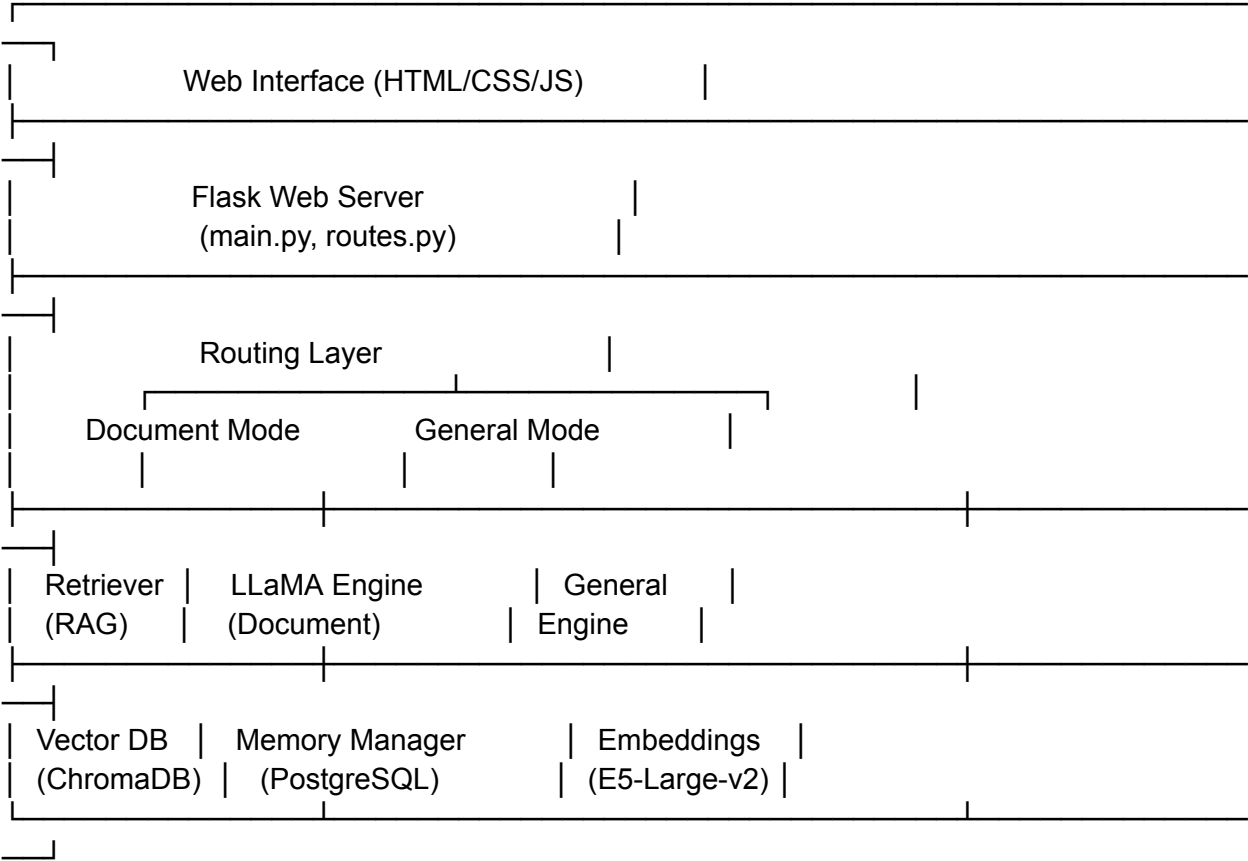
Frontend

- **HTML5/CSS3**: Modern responsive UI
- **JavaScript (ES6+)**: Interactive client-side logic
- **Marked.js**: Markdown parsing and rendering
- **Highlight.js 11.9.0**: Syntax highlighting for code blocks
- **Font Awesome 6.0.0**: Icon library

Utilities

- **tiktoken 0.5.2**: Token counting for LLMs
- **UUID**: Session identifier generation

System Architecture



Component Interaction Flow

1. **User Request** → Flask Routes (`routes.py`)
 2. **Session Management** → Memory Manager → PostgreSQL
 3. **Mode Detection** → Document/General routing
 4. **Document Mode:**
 - Retriever → Vector Store (ChromaDB)
 - Context Selection → LLaMA Document Engine
 5. **General Mode:**
 - Direct routing → LLaMA General Engine
 6. **Response Streaming** → Web Interface
 7. **History Storage** → PostgreSQL
-

Core Components

1. Main Application (`main.py`)

Purpose: Entry point for the Flask application

Key Functions:

- `index()`: Serves the frontend HTML
- `health_check()`: System health monitoring endpoint
- `get_stats()`: Returns retrieval system statistics
- `trigger_reindex()`: Manual document reindexing

Initialization:

```
app = Flask(__name__)
app.register_blueprint(routes)
retriever = Retriever()
```

2. Routes (`routes.py`)

Purpose: API endpoint handlers and request routing

Core Functions

`route_request(mode: str) -> str`

Determines routing strategy based on mode

- Input: `"document"` or `"general"`

- Returns: "two_step_rag" or "general"

`format_chat_history_for_prompt(history, max_messages=5) -> str`

Formats conversation history for context injection

- Extracts last N messages
- Formats as "User: ... / Assistant: ..." pairs
- Returns formatted string for prompt

`chat()` - Main Chat Endpoint

Flow:

1. Extract request parameters (message, mode, session_id, max_tokens)
2. Validate and generate UUID if needed
3. Save user message to PostgreSQL
4. Retrieve conversation history
5. Route to appropriate engine:
 - **Document Mode:** RAG pipeline with fallback
 - **General Mode:** Direct LLaMA inference
6. Stream response to client
7. Save AI response to database

Key Logic:

```
if engine_route == "two_step_rag":
    # Check document availability
    # Retrieve relevant chunks with scores
    # Check confidence threshold (0.50)
    if confidence < CONFIDENCE_THRESHOLD:
        # Fallback to general mode
    else:
        # Use document context
```

3. Configuration (`config.py`)

Purpose: Centralized configuration management

Key Settings:

```
class Config:
    # Paths
    PROJECT_ROOT = Path(__file__).parent
    DOCUMENTS_DIR = PROJECT_ROOT / "documents"
```

```
VECTOR_DB_DIR = PROJECT_ROOT / "data" / "vector_db"  
MODELS_DIR = PROJECT_ROOT / "models"
```

```
# Embedding Model  
EMBEDDING_MODEL = "models/e5-large-v2"  
EMBEDDING_DEVICE = "cpu"
```

```
# Chunking  
CHUNK_SIZE = 1000  
CHUNK_OVERLAP = 200  
SIMILARITY_THRESHOLD = 0.7
```

```
# Retrieval  
RERANK_K_CANDIDATES = 40  
TOP_K_RETRIEVAL = 8  
RERANKER_MODEL = "BAAI/bge-reranker-base"
```

```
# Server  
HOST = "127.0.0.1"  
PORT = 8800
```

4. Document Pipeline (**setup_pipeline.py**)

Purpose: Index documents into vector store

Main Function: **setup_rag_pipeline(force_reindex, validate_quality)**

Workflow:

1. Initialize components (EmbeddingManager, VectorStore)
2. Check existing index
3. Load documents from **./documents/**
4. Apply semantic chunking
5. Validate chunk quality (optional)
6. Add to ChromaDB
7. Test retrieval quality

Quality Validation:

- **validate_chunk_quality()**: Analyzes chunk statistics
- **test_retrieval_quality()**: Tests sample queries
- Reports: chunk sizes, semantic ratio, quality score

Backend Implementation

Document Loader (**document_loader.py**)

Expected Functions (based on usage):

- `load_documents()`: Loads all documents from directory
- `_load_pdf()`: PDF extraction
- `_load_docx()`: Word document extraction
- `_load_txt()`: Plain text loading
- `_load_markdown()`: Markdown parsing

Semantic Chunker (**semantic_chunker.py**)

Purpose: Intelligent document chunking using K-means clustering

Key Methods:

- `chunk_documents(documents)`: Main chunking entry point
- `_semantic_clusters(sentences, max_cluster_size)`: Groups semantically similar sentences
- Strategy: Clusters sentences by embedding similarity

Embedding Manager (**embedding_manager.py**)

Purpose: Text vectorization with E5-Large-v2

Expected Methods:

- `embed_text(text)`: Generate embeddings
- `embed_batch(texts)`: Batch embedding generation
- Uses HuggingFace sentence-transformers

Vector Store (**vector_store.py**)

Purpose: ChromaDB operations

Key Methods:

- `add_documents(documents)`: Index documents

- `similarity_search(query, k)`: Semantic search
- `get_collection_count()`: Count indexed documents
- `get_collection_stats()`: Statistics
- `clear_collection()`: Clear index

Retriever (`retriever.py`)

Purpose: Semantic retrieval with scoring

Pattern: Singleton for resource efficiency

Key Methods:

- `get_relevant_chunks(query, top_k)`: Basic retrieval
- `get_relevant_chunks_with_scores(query, top_k)`: With confidence scores
- `get_stats()`: Retriever statistics

Memory Manager (`memory_manager.py`)

Purpose: PostgreSQL-backed conversation persistence

Database Schema:

```
-- chat_history table
session_id UUID
message_id TEXT
role TEXT (user/assistant)
content TEXT
timestamp TIMESTAMP
```

```
-- session_metadata table
session_id UUID (PK)
title TEXT
created_at TIMESTAMP
updated_at TIMESTAMP
message_count INTEGER
```

Key Methods:

- `add_message(session_id, message, is_user)`: Store message
- `get_formatted_history(session_id, limit)`: Retrieve history
- `clear_session_history(session_id)`: Clear chat
- `delete_session(session_id)`: Delete session

- `get_all_sessions()`: List sessions
- `get_session_stats(session_id)`: Session statistics

LLaMA Engines

Document Engine (`llama_engine.py`)

Purpose: Document-specific responses

Key Components:

```
@lru_cache(maxsize=1)
def _load_llama() -> Llama:
    """Lazy load and cache model"""
    return Llama(
        model_path="models/Llama-3.2-1B-Instruct-Q5_K_L-Document-Model.gguf",
        n_ctx=4096,
        n_threads=max(os.cpu_count() - 4, 1),
        n_batch=512,
        chat_format="llama-2"
    )

def generate_stream(messages, max_tokens=2048):
    """Generate streaming response"""
    llama = _load_llama()
    for chunk in llama.create_chat_completion(
        messages=messages,
        max_tokens=max_tokens,
        temperature=0.6,
        stream=True
    ):
        delta = chunk["choices"][0]["delta"]
        if "content" in delta:
            yield delta["content"]
```

System Prompt: Loaded from `config/system_prompt.txt`

General Engine (`general_engine.py`)

Purpose: General conversation

Differences:

- Model: `Llama-3.2-1B-Instruct-Q5_K_L-General-Model.gguf`

- Temperature: 0.7 (more creative)
 - System Prompt: Loaded from `config/general_system_prompt`
-

Frontend Implementation

HTML Structure (`index.html`)

Layout:

```
<div class="app-container">
  <aside class="sidebar">
    <!-- Session list -->
  </aside>
  <main class="main-content">
    <header class="chat-header">
      <!-- Title, actions -->
    </header>
    <div class="chat-container">
      <div class="messages-container">
        <!-- Messages -->
      </div>
    </div>
    <div class="input-area">
      <!-- Input controls -->
    </div>
  </main>
</div>
```

External Libraries:

- Marked.js: Markdown rendering
- Highlight.js (Monokai theme): Code syntax highlighting
- Font Awesome: Icons

JavaScript Architecture

1. App Controller (`app.js`)

Class: `App`

Responsibilities: Main application orchestration

Key Properties:

```
{
  sessionManager: SessionManager,
  chatManager: ChatManager,
  currentSessionId: UUID,
  isMobile: boolean,
  isDocumentMode: boolean,
  settings: {
    maxTokens: 2048,
    userName: 'User',
    theme: 'light',
    enableSound: boolean,
    enableStreaming: boolean
  }
}
```

Key Methods:

sendMessage()

Flow:

1. Validate input
2. Add user message to UI
3. Save to session
4. Prepare request body
5. Fetch API with streaming
6. Process stream chunks
7. Update UI incrementally
8. Save final response

```
async sendMessage() {
  const requestBody = {
    message: message,
    session_id: this.currentSessionId,
    mode: this.isDocumentMode ? 'document' : 'general',
    max_tokens: this.settings.maxTokens
  };

  const response = await fetch('/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(requestBody)
```

```

});

const reader = response.body.getReader();
const decoder = new TextDecoder('utf-8');
let result = "";

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  const chunk = decoder.decode(value);
  result += chunk;
  this.chatManager.updateMessage(thinkingMsgId, result);
}
}

```

Other Methods:

- `createNewSession()`: Initialize new chat
- `switchToSession(sessionId)`: Load different session
- `deleteCurrentSession()`: Remove session
- `toggleDocumentMode()`: Switch modes
- `openSettings()`: Settings modal
- `clearCurrentChat()`: Clear messages
- `checkSystemStatus()`: Health check
- `showSystemStats()`: Display statistics

2. Chat Manager (`chatManager.js`)

Class: `ChatManager`

Responsibilities: Message display and chat UI

Key Methods:

`loadSession(session)`

Loads conversation into UI

- Filters out 'thinking' messages
- Normalizes sender names
- Renders each message
- Applies syntax highlighting

`addMessage(sender, content, animate)`

Creates new message bubble

- Generates message ID
- Adds to session
- Renders with animation
- Returns message object

updateMessage(messageId, newContent)

Updates streaming message

- Finds message element
- Updates sender from 'thinking' to 'bot'
- Renders markdown content
- Re-applies syntax highlighting

formatMessageContent(content)

Renders markdown with Marked.js

```
formatMessageContent(content) {  
  if (typeof marked !== 'undefined') {  
    return marked.parse(content);  
  }  
  return this.escapeHTML(content).replace(/\n/g, '<br>');  
}
```

applySyntaxHighlighting(messageId)

Applies Highlight.js to code blocks

```
applySyntaxHighlighting(messageId) {  
  const messageElement = document.querySelector(`[data-message-id="${messageId}"]`);  
  const codeBlocks = messageElement.querySelectorAll('pre code');  
  codeBlocks.forEach(el => hljs.highlightElement(el));  
}
```

addCopyButtonsToCode(container)

Adds language labels and copy buttons to code blocks

- Detects language from class
- Creates header with language name
- Adds copy button

- Handles clipboard operations

3. Session Manager (**sessionManager.js**)

Class: **SessionManager**

Storage: localStorage + PostgreSQL

Key Methods:

generateSessionId()

UUID v4 generation

```
generateSessionId() {
  return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, function(c) {
    const r = Math.random() * 16 | 0;
    const v = c == 'x' ? r : (r & 0x3 | 0x8);
    return v.toString(16);
  });
}
```

createSession(title)

Creates new session object

```
{
  id: UUID,
  title: string,
  messages: [],
  createdAt: ISO timestamp,
  updatedAt: ISO timestamp,
  metadata: {
    messageCount: 0,
    lastActivity: ISO timestamp
  }
}
```

generateTitleFromMessage(message)

Auto-generates title from first message

- Takes first 4 words
- Truncates to 30 characters

- Fallback: "New Chat"

`saveSessions()`

Persists to localStorage

- Handles quota errors
- Auto-cleanup old sessions (max 50)

Export Functions:

- `sessionToText(session)`: Plain text export
- `sessionToMarkdown(session)`: Markdown export

4. Utilities (`utils.js`)

Global Functions:

`showToast(message, type, duration)`

Toast notifications

- Types: info, success, error
- Auto-dismiss after duration
- Slide-in animation

`formatDate(dateString)`

Relative time formatting

- "just now", "Xm ago", "Xh ago"
- Falls back to absolute date

`escapeHTML(str)`

XSS protection

`copyToClipboard(text)`

Clipboard operations with fallback

`debounce(func, wait)`

Rate limiting for input handlers

`throttle(func, limit)`

Rate limiting for scroll handlers

Database Schema

PostgreSQL Tables

chat_history

```
CREATE TABLE chat_history (  
  id SERIAL PRIMARY KEY,  
  session_id UUID NOT NULL,  
  message_id TEXT NOT NULL,  
  role TEXT NOT NULL CHECK (role IN ('user', 'assistant')),  
  content TEXT NOT NULL,  
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  metadata JSONB  
);
```

```
CREATE INDEX idx_session_id ON chat_history(session_id);  
CREATE INDEX idx_timestamp ON chat_history(timestamp);
```

session_metadata

```
CREATE TABLE session_metadata (  
  session_id UUID PRIMARY KEY,  
  title TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  message_count INTEGER DEFAULT 0,  
  metadata JSONB  
);
```

```
CREATE INDEX idx_updated_at ON session_metadata(updated_at);
```

ChromaDB Collections

Collection: `documents` (configurable)

Document Structure:

```
{  
  "id": "unique_id",
```



```
"embedding": [float array],
"metadata": {
  "file_name": str,
  "chunk_method": "semantic_enhanced" | "fallback",
  "page": int,
  "source": str
},
"document": "chunk text content"
}
```

API Endpoints

Chat Endpoint

POST /chat

Content-Type: application/json

Request:

```
{
  "message": "User query",
  "session_id": "UUID",
  "mode": "document" | "general",
  "max_tokens": 2048
}
```

Response: text/plain (streaming)

Memory Endpoints

Get Session History

GET /memory/history/<session_id>?limit=50

Response:

```
{
  "session_id": "UUID",
  "message_count": int,
  "messages": [
    {
      "role": "user" | "assistant",
      "content": "message text",
      "timestamp": "ISO timestamp"
    }
  ]
}
```

```
    }  
  ]  
}
```

Clear Session History

POST /memory/clear/<session_id>

Response:

```
{  
  "message": "History cleared",  
  "session_id": "UUID"  
}
```

Delete Session

DELETE /memory/delete/<session_id>

Response:

```
{  
  "message": "Session deleted",  
  "session_id": "UUID"  
}
```

List All Sessions

GET /memory/sessions

Response:

```
{  
  "session_count": int,  
  "sessions": [  
    {  
      "session_id": "UUID",  
      "title": "Session title",  
      "created_at": "ISO timestamp",  
      "updated_at": "ISO timestamp",  
      "message_count": int  
    }  
  ]  
}
```

Get Session Stats

GET /memory/stats/<session_id>

Response:

```
{
  "total_messages": int,
  "user_messages": int,
  "bot_messages": int,
  "created_at": "ISO timestamp",
  "updated_at": "ISO timestamp"
}
```

System Endpoints

Health Check

GET /health

Response:

```
{
  "status": "healthy" | "unhealthy",
  "retriever_stats": {
    "total_documents": int,
    "embedding_model": str
  },
  "timestamp": float
}
```

Statistics

GET /stats

Response:

```
{
  "total_documents": int,
  "total_chunks": int,
  "vector_store_path": str
}
```

Reindex Documents

POST /reindex

Response:

```
{
```

```
"message": "Reindexing completed",
"documents_processed": int,
"chunks_created": int
}
```

Key Functions & Methods

Backend Critical Functions

routes.py

CONFIDENCE_THRESHOLD: 0.50

- Minimum relevance score for document mode
- Below threshold triggers fallback to general mode

Session ID Validation:

```
try:
    uuid.UUID(session_id)
except ValueError:
    session_id = str(uuid.uuid4())
```

History Integration:

```
history_context = format_chat_history_for_prompt(chat_history)
enhanced_question = f"""Previous conversation:
{history_context}
```

```
Current question: {question}"""
```

Streaming Response Pattern:

```
accumulated_response = "".join(token for token in llama_stream(messages))
memory_manager.add_message(session_id, accumulated_response, is_user=False)
```

```
def stream_to_client():
    for char in accumulated_response:
        yield char
```

```
return Response(stream_to_client(), mimetype="text/plain; charset=utf-8")
```

Frontend Critical Functions

Message Rendering Pipeline

1. **User Input** → `app.sendMessage()`
2. **Add to UI** → `chatManager.addMessage('user', message)`
3. **Streaming** → `fetch('/chat')` with `ReadableStream`
4. **Update UI** → `chatManager.updateMessage(id, chunk)`
5. **Markdown** → `marked.parse(content)`
6. **Highlighting** → `hljs.highlightElement(codeBlock)`
7. **Save** → `sessionManager.addMessageToSession()`

Persistence Strategy

Dual Storage:

1. **localStorage**: Client-side cache for quick loading
2. **PostgreSQL**: Server-side persistence for durability

Sync Points:

- On message send: Immediate save to both
 - On session switch: Save current, load new
 - On page unload: Final save via `beforeunload` event
-

Data Flow

Document Query Flow

1. User Query
↓
2. `routes.py:/chat (mode="document")`
↓
3. `retriever.get_relevant_chunks_with_scores(query, top_k=8)`
↓
4. ChromaDB similarity search
↓
5. Score filtering (threshold=0.50)
↓
6. If score ≥ 0.50 :

└─> context = relevant_chunks
↓
llama_engine.generate_stream([system, user+context])

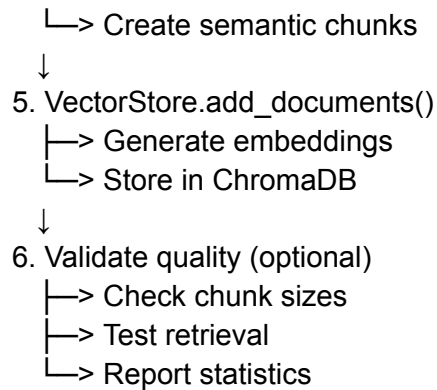
If score < 0.50:
└─> Fallback to general_engine
↓
general_engine.generate_stream([system, user])
↓
7. Stream response to client
↓
8. memory_manager.add_message() → PostgreSQL

General Query Flow

1. User Query
↓
2. routes.py:/chat (mode="general")
↓
3. memory_manager.get_formatted_history(session_id)
↓
4. Enhance query with history context
↓
5. general_engine.generate_stream([system, user+history])
↓
6. Stream response to client
↓
7. memory_manager.add_message() → PostgreSQL

Document Indexing Flow

1. Place documents in ./documents/
↓
2. Run: python setup_pipeline.py
↓
3. DocumentLoader.load_documents()
└─> PDF: PyPDF/PyMuPDF
└─> DOCX: python-docx
└─> TXT: Direct read
↓
4. SemanticChunker.chunk_documents()
└─> Embed sentences (E5-Large-v2)
└─> K-means clustering



Configuration

Environment Variables (Optional)

```
POSTGRES_CONNECTION_STRING="postgresql://user:pass@localhost:5432/dbname"
SESSION_SECRET="your-secret-key"
```

Model Configuration

Document Model:

- Path: `models/Llama-3.2-1B-Instruct-Q5_K_L-Document-Model.gguf`
- Context: 4096 tokens
- Temperature: 0.6
- Batch: 512

General Model:

- Path: `models/Llama-3.2-1B-Instruct-Q5_K_L-General-Model.gguf`
- Context: 4096 tokens
- Temperature: 0.7
- Batch: 512

Retrieval Configuration

```
CHUNK_SIZE = 1000      # Characters per chunk
CHUNK_OVERLAP = 200    # Overlap between chunks
SIMILARITY_THRESHOLD = 0.7 # For semantic clustering
CONFIDENCE_THRESHOLD = 0.50 # For document mode routing
TOP_K_RETRIEVAL = 8     # Final results to return
RERANK_K_CANDIDATES = 40 # Initial candidates before reranking
```

Installation & Setup

Prerequisites

- Python 3.8+
- PostgreSQL 12.0+
- 8GB+ RAM
- 10GB storage

Steps

1. Clone & Setup Environment

```
git clone <repository-url>
cd ViduraAI
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

2. PostgreSQL Setup

```
CREATE DATABASE vidura_ai;
CREATE USER postgres WITH PASSWORD 'your_password';
GRANT ALL PRIVILEGES ON DATABASE vidura_ai TO postgres;
```

3. Update Connection String Edit `src/memory_manager.py` line 32:

```
connection_string = "postgresql://postgres:your_password@localhost:5432/vidura_ai"
```

4. Download Models

- Place LLaMA models in `./models/`
- E5-Large-v2 will auto-download on first run

5. Add Documents

```
mkdir -p documents/
cp /path/to/your/*.pdf ./documents/
```

6. Index Documents

python setup_pipeline.py

7. Start Server

python main.py

8. Access Application

http://127.0.0.1:8800

Performance Considerations

Backend Optimization

- **Model Caching:** `@lru_cache` for singleton pattern
- **Lazy Loading:** Models loaded on first request
- **Thread Configuration:** `n_threads = max(cpu_count - 4, 1)`
- **Batch Processing:** `n_batch = 512`

Frontend Optimization

- **Incremental Rendering:** Stream chunks as they arrive
- **Virtual Scrolling:** Only render visible messages
- **Debouncing:** Input handlers throttled
- **Code Splitting:** Separate CSS files

Database Optimization

- Indexes on `session_id`, `timestamp`
- Connection pooling (SQLAlchemy)
- Query result caching

Vector Store Optimization

- HNSW parameters tuned in ChromaDB
 - Metadata filtering for faster search
 - Batch embedding generation
-

Security Features

- **Local Processing:** All LLM inference runs locally
 - **No External APIs:** Document content never leaves server
 - **Session Isolation:** UUID-based session management
 - **XSS Protection:** `escapeHTML()` on user content
 - **SQL Injection Protection:** SQLAlchemy parameterized queries
 - **CSRF Protection:** Flask secret key for session security
-

Error Handling

Backend

- Try-catch blocks on all API endpoints
- Graceful degradation (RAG → General fallback)
- Detailed logging with session IDs
- HTTP status codes (400, 500, 503)

Frontend

- Network error handling in `sendMessage()`
 - Toast notifications for user feedback
 - Modal confirmations for destructive actions
 - Fallback rendering for markdown failures
-

Troubleshooting

Common Issues

1. PostgreSQL Connection Error

```
# Check PostgreSQL status
sudo systemctl status postgresql
# Test connection
psql -U postgres -d vidura_ai
```

2. Model Not Found

```
# Verify model files
ls -la models/*.gguf
```

3. Out of Memory

```
# Reduce batch size in config
n_batch = 128 # Instead of 512
```

4. Documents Not Indexing

```
# Check permissions
ls -la documents/
# Verify formats
file documents/*
```

Future Enhancements

1. **Multi-Model Support:** Add support for different LLM backends (OpenAI, Anthropic, etc.)
 2. **Distributed Search:** Implement distributed vector search for scaling
 3. **Caching Layer:** Add Redis for response caching and rate limiting
 4. **API Gateway:** Implement authentication and API key management
 5. **Monitoring:** Add Prometheus metrics and Grafana dashboards
 6. **Hybrid Search:** Combine semantic search with keyword-based retrieval
 7. **Multi-Language Support:** Internationalization for UI and responses
 8. **Voice Input:** Speech-to-text integration
 9. **File Upload:** Direct document upload through UI
 10. **Export Features:** Download conversations as PDF/DOCX
-

Package Dependencies Breakdown

Core Framework & Server

```
flask                # Web framework for routing and API
uvicorn[standard]    # ASGI server (not actively used, Flask uses WSGI)
```

LangChain Ecosystem

```
langchain==0.1.0      # Document processing framework
```

langchain-community==0.0.13 # Community integrations
langchain-postgres # PostgreSQL integration for chat history

Vector Database & Embeddings

chromadb==0.4.22 # Vector database for semantic search
sentence-transformers==3.0.1 # Modern embedding models (E5-Large-v2)

Document Processing

pypdf2==3.0.1 # PDF extraction
pypdf==6.0.0 # Alternative PDF parser
PyMuPDF # Advanced PDF processing (fitz library)
python-docx==1.1.0 # Word document processing
docx2txt==0.9 # Text extraction from DOCX

Machine Learning & Deep Learning

torch==2.1.2 # PyTorch for deep learning models
transformers==4.36.2 # HuggingFace transformers library
accelerate==0.25.0 # Efficient model loading and inference
bitsandbytes==0.42.0 # Model quantization support

LLM Inference

llama_cpp_python # Python bindings for llama.cpp (GGUF models)
tiktoken==0.5.2 # Token counting for LLMs

Database & ORM

sqlalchemy # ORM for database operations
psycopg[binary] # PostgreSQL adapter for Python

Data Processing

pandas==2.1.4 # Data manipulation and analysis
numpy==1.24.3 # Numerical computing

NLP & Text Processing

NLTK # Natural language toolkit

huggingface_hub==0.20.3 # HuggingFace model hub integration

Detailed Function Reference

Backend Functions

routes.py Functions

route_request(mode: str) -> str

```
def route_request(mode: str) -> str:
    """
    Determine routing strategy based on mode
```

Args:

mode: "document" or "general"

Returns:

"two_step_rag" for document mode

"general" for general mode

```
"""
```

```
    return "two_step_rag" if mode == "document" else "general"
```

format_chat_history_for_prompt(history: List[Dict], max_messages: int = 5) -> str

```
def format_chat_history_for_prompt(history: List[Dict[str, str]], max_messages: int = 5) -> str:
    """
```

Format recent chat history into a string for context injection

Args:

history: List of message dicts with 'role' and 'content'

max_messages: Maximum number of recent messages to include

Returns:

Formatted string with "User: ... \n Assistant: ..." format

```
"""
```

```
if not history:
```

```
    return ""
```

```
recent = history[-max_messages:] if len(history) > max_messages else history
```

```
formatted_lines = []
```

```

for msg in recent:
    role = "User" if msg["role"] == "user" else "Assistant"
    formatted_lines.append(f"{role}: {msg['content']}")

return "\n".join(formatted_lines)

```

chat() - POST /chat

Purpose: Main chat endpoint handling all user queries

Request Parameters:

- **message** (str): User's query text
- **mode** (str): "document" or "general"
- **session_id** (UUID): Session identifier
- **max_tokens** (int): Maximum tokens in response (default: 2048)

Response: text/plain stream (UTF-8 encoded)

Key Steps:

1. **Validation:** Check session_id is valid UUID
2. **Save User Message:** Store in PostgreSQL before processing
3. **Retrieve History:** Get last 5 messages for context
4. **Routing Decision:**
 - Document mode → Check document availability → RAG pipeline
 - General mode → Direct to general engine
5. **RAG Pipeline** (if document mode):
 - Retrieve top 8 relevant chunks with scores
 - Check confidence threshold (0.50)
 - If score ≥ 0.50 : Use document context
 - If score < 0.50 : Fallback to general mode
6. **Generate Response:** Stream from appropriate engine
7. **Save AI Response:** Store complete response in PostgreSQL
8. **Stream to Client:** Yield response character by character

Error Handling:

- Empty message → 400 Bad Request
- Database errors → Logged, continue with empty history
- Model errors → 500 Internal Server Error

Memory Endpoints

GET /memory/history/<session_id>

- Returns formatted conversation history
- Optional `limit` query parameter (default: 50)
- Validates UUID format

POST /memory/clear/<session_id>

- Clears all messages in a session
- Preserves session metadata
- Validates UUID format

DELETE /memory/delete/<session_id>

- Deletes session and all associated data
- Removes from both `session_metadata` and `chat_history` tables
- Validates UUID format

GET /memory/sessions

- Lists all sessions
- Returns session metadata (title, timestamps, message count)
- Sorted by `updated_at` descending

GET /memory/stats/<session_id>

- Returns session statistics
- Message counts (total, user, bot)
- Timestamps and duration

Memory Manager Methods

`add_message(session_id: str, message: str, is_user: bool = True)`

`def add_message(self, session_id: str, message: str, is_user: bool = True):`

"""

Add a message to the chat history with explicit commit

Args:

session_id: UUID of the session

message: Message content

is_user: True for user messages, False for AI responses

Side Effects:

- Inserts into `chat_history` table

- Updates `session_metadata` `message_count` and `updated_at`

- Commits transaction immediately

"""

Transaction Pattern:

```
try:
    # Insert message
    session.add(message_record)

    # Update session metadata
    session_meta.message_count += 1
    session_meta.updated_at = datetime.now()

    # Explicit commit
    session.commit()
except Exception as e:
    session.rollback()
    raise
```

get_formatted_history(session_id: str, limit: int = 10) -> List[Dict]

def get_formatted_history(self, session_id: str, limit: int = 10) -> List[Dict]:

"""

Retrieve conversation history formatted for UI or prompts

Args:

session_id: UUID of the session

limit: Maximum number of messages to retrieve

Returns:

List of message dictionaries:

```
[
    {
        "role": "user" | "assistant",
        "content": "message text",
        "timestamp": "ISO format"
    }
]
```

Notes:

- Ordered by timestamp ascending

- Limited to most recent N messages

"""

clear_session_history(session_id: str)

Deletes all messages for a session while preserving session metadata

delete_session(session_id: str)

Completely removes session and all associated data

get_all_sessions() -> List[Dict]

Returns list of all sessions with metadata

get_session_stats(session_id: str) -> Dict

Returns statistics for a specific session

Frontend Functions

App.js Methods

sendMessage()

Purpose: Handle user message submission and response streaming

Flow:

1. Validate input (non-empty)
2. Ensure session exists (create if needed)
3. Add user message to UI
 - chatManager.addMessage('user', message, true)
4. Add user message to session
 - sessionManager.addMessageToSession(sessionId, message)
5. Clear input and update UI
6. Add "Thinking..." placeholder
 - chatManager.addMessage('thinking', 'Thinking...', true)
7. Prepare request body
8. Fetch /chat endpoint
9. Get readable stream reader
10. Stream processing loop:
 - while (true):
 - Read chunk from stream
 - Decode UTF-8
 - Append to result
 - Update UI incrementally (chatManager.updateMessage)
11. Apply syntax highlighting to final message
12. Save AI response to session
13. Update session title if first message
14. Refresh session list

Error Handling:

```
try {  
    // Streaming logic  
} catch (error) {  
    console.error('Error in sendMessage:', error);  
    chatManager.updateMessage(thinkingMsgId, `Error: ${error.message}`);  
    showToast('Failed to send message', 'error');  
}
```

createNewSession()

```
createNewSession() {  
    const session = this.sessionManager.createSession();  
    this.switchToSession(session.id);  
    this.updateSessionsList();  
  
    // Auto-focus input  
    setTimeout(() => {  
        document.getElementById('messageInput').focus();  
    }, 100);  
  
    showToast('New session created', 'success');  
}
```

switchToSession(sessionId)

```
switchToSession(sessionId) {  
    if (this.currentSessionId === sessionId) return;  
  
    // Save current session before switching  
    if (this.currentSessionId) {  
        this.saveCurrentSession();  
    }  
  
    this.currentSessionId = sessionId;  
    const session = this.sessionManager.getSession(sessionId);  
  
    if (session) {  
        this.chatManager.loadSession(session);  
        this.updateUI();  
        this.closeMobileSidebar();  
    }  
}
```

deleteCurrentSession() / deleteSpecificSession(sessionId)

Purpose: Delete session from both backend and frontend

Flow:

1. Show confirmation modal with message count
2. On confirm:
 - a. Call DELETE /memory/delete/<session_id>
 - b. Check response status
 - c. If successful:
 - Delete from sessionManager (localStorage)
 - Switch to another session or create new one
 - Update session list
 - Show success toast
 - d. If failed:
 - Log error
 - Show error toast

clearCurrentChat()

Purpose: Clear all messages in current session

Flow:

1. Check message count
2. Show confirmation modal
3. On confirm:
 - a. Call POST /memory/clear/<session_id>
 - b. Check response status
 - c. If successful:
 - Clear messages array in sessionManager
 - Reload session in chatManager
 - Update UI
 - Show success toast
 - d. If failed:
 - Log error
 - Show error toast

toggleDocumentMode()

```
toggleDocumentMode() {  
    const docBtn = document.getElementById('documentBtn');
```

```

this.isDocumentMode = !this.isDocumentMode;

// Update button appearance
if (this.isDocumentMode) {
  docBtn.classList.add('btn-primary', 'active');
} else {
  docBtn.classList.remove('btn-primary', 'active');
}

// Update placeholder text
const messageInput = document.getElementById('messageInput');
if (this.isDocumentMode) {
  messageInput.placeholder = 'Ask me anything about your documents...';
} else {
  messageInput.placeholder = 'Message AI Assistant...';
}
}

```

openSettings() / saveSettings()

Purpose: Manage user preferences

Settings Stored:

```

{
  userName: string,
  maxTokens: number (50-2048),
  theme: 'light' | 'dark' | 'auto',
  enableSound: boolean,
  enableStreaming: boolean
}

```

Storage: localStorage key 'chatbot_settings'

checkSystemStatus()

Purpose: Check backend health and document index status

```

async checkSystemStatus() {
  try {
    const response = await fetch('/health');
    const data = await response.json();

```

```

const statusEl = document.getElementById('systemStatus');
if (data.status === 'healthy') {
  const docCount = data.retriever_stats?.total_documents || 0;
  statusEl.textContent = `✓ ${docCount} documents indexed`;
  statusEl.style.color = 'var(--accent-primary)';
} else {
  statusEl.textContent = '⚠ Not ready';
  statusEl.style.color = '#f59e0b';
}
} catch (error) {
  const statusEl = document.getElementById('systemStatus');
  statusEl.textContent = '✗ Offline';
  statusEl.style.color = 'var(--danger)';
}
}

```

ChatManager.js Methods

loadSession(session)

```

loadSession(session) {
  // Deep clone to avoid reference issues
  this.currentSession = {
    ...session,
    messages: session.messages.map(msg => ({ ...msg }))
  };

  this.clearMessages();

  if (this.currentSession.messages.length > 0) {
    // Filter out thinking messages
    const cleanedMessages = this.currentSession.messages
      .filter(msg => msg.sender !== 'thinking');

    this.currentSession.messages = cleanedMessages;

    cleanedMessages.forEach(message => {
      // Ensure message has ID
      if (!message.id) message.id = this.generateMessageId();

      // Normalize sender names
      if (message.sender === 'assistant' || message.sender === 'ai') {
        message.sender = 'bot';
      }
    });
  }
}

```

```

        // Display without animation
        this.displayMessage(message, false);

        // Apply syntax highlighting to code blocks
        if (message.sender === 'bot') {
            this.applySyntaxHighlighting(message.id);
        }
    });

    this.scrollToBottom();
} else {
    this.showEmptyState();
}
}

```

addMessage(sender, content, animate = true)

```

addMessage(sender, content, animate = true) {
    if (!this.currentSession) return null;

    const safeContent = typeof content === "string" ? content : String(content || "");

    const message = {
        id: this.generateMessageId(),
        sender,
        content: safeContent,
        timestamp: new Date().toISOString(),
        metadata: {
            wordCount: safeContent.split(/\s+/).length,
            characterCount: safeContent.length
        }
    };

    if (!this.currentSession.messages) {
        this.currentSession.messages = [];
    }

    this.currentSession.messages.push(message);
    this.hideEmptyState();

    const messageElement = this.displayMessage(message, animate);

    // Only apply highlighting if it's a final message

```

```

    if (message.sender === 'user' || message.sender === 'bot') {
      this.applySyntaxHighlighting(message.id);
    }

    this.scrollToBottom();

    return { message, messageElement };
  }

```

updateMessage(messageId, newContent)

Purpose: Update message during streaming

```

updateMessage(messageId, newContent) {
  if (!this.currentSession || !this.currentSession.messages) return;

  // Update message in data model
  const messageIndex = this.currentSession.messages
    .findIndex(msg => msg.id === messageId);

  if (messageIndex !== -1) {
    // Change 'thinking' to 'bot'
    if (this.currentSession.messages[messageIndex].sender === 'thinking') {
      this.currentSession.messages[messageIndex].sender = 'bot';
    }
    this.currentSession.messages[messageIndex].content = newContent;
  }

  // Update DOM element
  const messageElement = document.querySelector('[data-message-id="${messageId}"]');
  if (messageElement) {
    // Update classes
    if (messageElement.classList.contains('thinking')) {
      messageElement.classList.replace('thinking', 'bot');
    }

    // Update avatar
    const avatar = messageElement.querySelector('.message-avatar');
    if (avatar && avatar.classList.contains('thinking')) {
      avatar.classList.remove('thinking');
      avatar.textContent = 'A';
      avatar.title = 'AI Assistant';
    }
  }
}

```

```

    // Update content bubble
    const bubble = messageElement.querySelector('.message-bubble');
    if (bubble) {
        bubble.classList.remove('thinking-bubble');

        // Render markdown
        bubble.innerHTML = this.formatMessageContent(newContent);

        // Re-add copy buttons
        this.addCopyButtonsToCode(bubble);
    }
}

this.scrollToBottom();
}

```

formatMessageContent(content)

```

formatMessageContent(content) {
    if (typeof marked !== 'undefined') {
        // Use Marked.js for markdown parsing
        return marked.parse(content);
    } else {
        // Fallback: escape HTML and convert newlines
        let escaped = this.escapeHTML(content);
        return escaped.replace(/\n/g, '<br>');
    }
}

```

applySyntaxHighlighting(messageId)

```

applySyntaxHighlighting(messageId) {
    if (typeof hljs === 'undefined') return;

    const messageElement = document.querySelector(`[data-message-id="${messageId}"]`);
    if (messageElement) {
        const codeBlocks = messageElement.querySelectorAll('pre code');
        codeBlocks.forEach(el => hljs.highlightElement(el));
    }
}

```

addCopyButtonsToCode(container)

Purpose: Add language labels and copy buttons to code blocks


```

addCopyButtonsToCode(container = this.messageContainer) {
  const codeBlocks = container.querySelectorAll('pre code');

  codeBlocks.forEach(codeBlock => {
    const pre = codeBlock.parentElement;

    // Skip if already has actions
    if (pre.querySelector('.code-actions')) return;

    // Detect language
    let language = 'text';
    const classMatch = codeBlock.className.match(/language-(\w+)/);
    if (classMatch && classMatch[1]) {
      language = classMatch[1];
      language = language.charAt(0).toUpperCase() + language.slice(1);
    }

    // Create actions container
    const actionsDiv = document.createElement('div');
    actionsDiv.className = 'code-actions';

    // Language label
    const langLabel = document.createElement('span');
    langLabel.className = 'code-lang-label';
    langLabel.textContent = language;

    // Copy button
    const copyBtn = document.createElement('button');
    copyBtn.className = 'btn btn-sm btn-outline-secondary code-copy-btn';
    copyBtn.innerHTML = '<i class="fas fa-copy"></i>';
    copyBtn.title = 'Copy code';

    actionsDiv.appendChild(langLabel);
    actionsDiv.appendChild(copyBtn);

    pre.style.position = 'relative';
    pre.appendChild(actionsDiv);

    // Copy event handler
    copyBtn.addEventListener('click', async () => {
      const code = codeBlock.textContent;
      const success = await this.copyToClipboard(code);

      if (success) {

```

```

        const icon = copyBtn.querySelector('i');
        const originalClass = icon.className;
        icon.className = 'fas fa-check';
        setTimeout(() => icon.className = originalClass, 1000);
        this.showToast('Code copied to clipboard', 'success');
    }
    });
}

```

SessionManager.js Methods

generateSessionId()

Purpose: Generate UUID v4 for session identification

```

generateSessionId() {
    // Browser-compatible UUID v4 generator
    return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, function(c) {
        const r = Math.random() * 16 | 0;
        const v = c == 'x' ? r : (r & 0x3 | 0x8);
        return v.toString(16);
    });
}

```

isValidUUID(str)

```

isValidUUID(str) {
    const uuidRegex = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
    return uuidRegex.test(str);
}

```

createSession(title = null)

```

createSession(title = null) {
    const sessionId = this.generateSessionId();

    const session = {
        id: sessionId,
        title: title || this.generateSessionTitle(),
        messages: [],
        createdAt: new Date().toISOString(),
        updatedAt: new Date().toISOString(),
        metadata: {

```

```

        messageCount: 0,
        lastActivity: new Date().toISOString()
    }
};

this.sessions.set(sessionId, session);
this.saveSessions();

console.log('Created new session:', sessionId);
return session;
}

```

generateSessionTitle()

```

generateSessionTitle() {
    const now = new Date();
    const timeString = now.toLocaleTimeString('en-US', {
        hour: 'numeric',
        minute: '2-digit',
        hour12: true
    });
    const dateString = now.toLocaleDateString('en-US', {
        month: 'short',
        day: 'numeric'
    });

    return `Chat ${dateString} ${timeString}`;
}

```

generateTitleFromMessage(message)

```

generateTitleFromMessage(message) {
    // Clean and truncate the message for title
    const cleaned = message.replace(/[\^\w\s]/g, "").trim();
    const words = cleaned.split(/\s+/).slice(0, 4); // First 4 words
    let title = words.join(' ');

    if (title.length > 30) {
        title = title.substring(0, 27) + '...';
    }

    return title || 'New Chat';
}

```

saveSessions()

Purpose: Persist sessions to localStorage with error handling

```
saveSessions() {
  try {
    const data = {
      version: '1.0',
      savedAt: new Date().toISOString(),
      sessions: Array.from(this.sessions.values())
    };

    localStorage.setItem(this.storageKey, JSON.stringify(data));
  } catch (error) {
    console.error('Failed to save sessions:', error);

    // Handle QuotaExceededError
    if (error.name === 'QuotaExceededError') {
      this.cleanupOldSessions(20);

      // Retry save
      try {
        const data = {
          version: '1.0',
          savedAt: new Date().toISOString(),
          sessions: Array.from(this.sessions.values())
        };
        localStorage.setItem(this.storageKey, JSON.stringify(data));
      } catch (retryError) {
        console.error('Failed to save sessions after cleanup:', retryError);
      }
    }
  }
}
```

cleanupOldSessions(maxSessions = 50)

```
cleanupOldSessions(maxSessions = 50) {
  const allSessions = this.getAllSessions();

  if (allSessions.length <= maxSessions) return;

  // Sort by update time, keep most recent
  allSessions.sort((a, b) => new Date(b.updatedAt) - new Date(a.updatedAt));
```

```

const sessionsToDelete = allSessions.slice(maxSessions);
let deletedCount = 0;

sessionsToDelete.forEach(session => {
  if (this.sessions.delete(session.id)) {
    deletedCount++;
  }
});

if (deletedCount > 0) {
  this.saveSessions();
  console.log(`Cleaned up ${deletedCount} old sessions`);
}

return deletedCount;
}

```

Export Functions

sessionToText(session)

```

sessionToText(session) {
  let text = `Chat Session: ${session.title}\n`;
  text += `Created: ${new Date(session.createdAt).toLocaleString()}\n`;
  text += `Updated: ${new Date(session.updatedAt).toLocaleString()}\n`;
  text += `Messages: ${session.messages.length}\n\n`;
  text += '---\n\n';

  session.messages.forEach(message => {
    const timestamp = new Date(message.timestamp).toLocaleTimeString();
    const sender = message.sender === 'user' ? 'You' : 'Bot';
    text += `[${timestamp}] ${sender}: ${message.content}\n\n`;
  });

  return text;
}

```

sessionToMarkdown(session)

```

sessionToMarkdown(session) {
  let markdown = `# ${session.title}\n\n`;
  markdown += `**Created:** ${new Date(session.createdAt).toLocaleString()}\n`;
  markdown += `**Updated:** ${new Date(session.updatedAt).toLocaleString()}\n`;
}

```

```
markdown += `**Messages:** ${session.messages.length}\n\n`;
markdown += '---\n\n';

session.messages.forEach(message => {
  const timestamp = new Date(message.timestamp).toLocaleTimeString();
  const sender = message.sender === 'user' ? '**You**' : '**Bot**';
  markdown += `### ${sender} *(${timestamp})*\n\n${message.content}\n\n`;
});

return markdown;
}
```

CSS Architecture

Design System

Color Palette (Light Theme):

```
--bg-primary: #ffffff      /* Main background */
--bg-secondary: #f7f7f8    /* Secondary surfaces */
--bg-tertiary: #ececfc      /* Tertiary surfaces */
--bg-hover: #f0f0f0        /* Hover states */

--text-primary: #2d2d2d     /* Primary text */
--text-secondary: #6e6e80   /* Secondary text */
--text-tertiary: #8e8ea0    /* Tertiary text */

--accent-primary: #10a37f   /* Primary accent (green) */
--accent-hover: #0d8a6a     /* Accent hover */
--accent-light: #e6f7f4     /* Accent light */

--danger: #ef4444          /* Danger/delete */
```

Layout Variables:

```
--sidebar-width: 260px
--header-height: 64px
--radius-sm: 6px
--radius-md: 8px
--radius-lg: 12px
--radius-xl: 16px
```

Key CSS Features

Flexbox Layout:

```
.app-container {  
  display: flex;          /* Sidebar + Content side-by-side */  
  height: 100vh;  
  overflow: hidden;  
}  
  
.main-content {  
  flex-grow: 1;          /* Fill remaining space */  
  display: flex;  
  flex-direction: column; /* Stack header, chat, input */  
}  
  
.chat-container {  
  flex: 1;               /* Fill vertical space */  
  overflow-y: auto;       /* Scrollable messages */  
}
```

Message Styling:

- User messages: Right-aligned, accent color
- Bot messages: Left-aligned, secondary background
- Thinking indicator: Animated dots
- Code blocks: Monokai theme with language labels

Animations:

- `messageSlideIn`: Fade in + slide up
- `thinkingPulse`: Opacity pulse for dots
- `typingDot`: Bounce animation for typing indicator
- `slideInLeft`: Session list item entrance

Responsive Design:

- Desktop: Sidebar always visible
 - Mobile (< 768px): Sidebar becomes drawer with overlay
 - Touch-friendly button sizes (minimum 36px)
-

Deployment Considerations

WSGI Configuration (**wsgi.py**)

```
from asgiref.wsgi import WsgiToAsgi
from main import app
```

```
# Create WSGI-compatible wrapper
application = WsgiToAsgi(app)
```

Usage with Gunicorn:

```
gunicorn -w 4 -b 0.0.0.0:8800 wsgi:application
```

Production Checklist

1. Environment Variables:

- Set production **SESSION_SECRET**
- Configure **POSTGRES_CONNECTION_STRING**

2. Security:

- Enable HTTPS
- Set secure session cookies
- Implement rate limiting
- Add CORS headers if needed

3. Performance:

- Enable model caching
- Configure connection pooling
- Set up Redis for session storage
- Implement response caching

4. Monitoring:

- Add logging aggregation (ELK stack)
- Set up metrics collection (Prometheus)
- Configure alerts for errors
- Monitor model inference times

5. Backup:

- Regular PostgreSQL backups
- ChromaDB snapshot backups

- Session export functionality
-

Development Workflow

Adding a New Feature

1. Backend:

- Add function to appropriate module (`src/`)
- Update `routes.py` if new endpoint needed
- Test with curl/Postman
- Update documentation

2. Frontend:

- Add UI elements to `index.html`
- Add styles to appropriate CSS file
- Implement logic in relevant JS class
- Test in browser console
- Update documentation

3. Testing:

- Unit tests for backend functions
- Integration tests for API endpoints
- Manual testing of UI interactions
- Cross-browser compatibility check

Debugging Tips

Backend:

```
# Enable verbose logging
import logging
logging.basicConfig(level=logging.DEBUG)

# Monitor database queries
engine = create_engine(connection_string, echo=True)

# Profile performance
import cProfile
profiler = cProfile.Profile()
profiler.enable()
```

```
# ... your code ...  
profiler.disable()  
stats = pstats.Stats(profiler)  
stats.print_stats()
```

Frontend:

```
// Enable console logging
```